CRICLAB

THE COMPREHENSIVE SYSTEM MANUAL & MANUAL TUTORIAL

An Offline-First Native Android Cricket Scoring & Management Suite

Technical Stack & Architecture Specifications

Scoring Engine V2 User & Admin Manual

Lifecycle Replay and Offline Synchronization Guide

Release Date: June 20, 2026

System Version: v2.1.0 (Stable release)

1. Executive Summary & Overview

1.1 Introduction

CricLab is a next-generation, high-performance, offline-first cricket scoring and player career management application. Designed specifically to work seamlessly in settings with variable network connectivity, CricLab encapsulates a full-featured web dashboard and a native Android container compiled via Capacitor JS.

By combining local relational persistence (SQLite) with background API replication, CricLab ensures that match scorers can operate fully offline on the field, without risking data loss, UI latencies, or app crashes. Once network access is restored, the local databases synchronize automatically with the central servers.

1.2 Architectural Philosophy

The core design of CricLab revolves around three architectural pillars:

- Offline-First Autonomy: The application runs its entire scoring state machine locally on the device using SQLite. Scorer act ions bypass remote network requests entirely, eliminating scoring button latency.

- Event-Stream Immutability: Matches are recorded as a chronological feed of immu table ball events (e.g. runs, wickets, extras). The scoreboard statistics are dynamically generated from this log, enabling error-free undo and correction replays.

- Dec

oupled Domain Model: Player management, squad rosters, match-squad assignments, and active innings parameters are stored as independent entities. Changes to tea ms

or global rosters never retrospectively alter historical scorecards.

2. Complete Technical Stack

2.1 Frontend & Application Layer

CricLab's UI is constructed as a modern, reactive single-page app containing these core technologies:

- React 19 (TypeScript): The baseline user interface library utilizing strict type contracts for component states, rendering, and API payloads.
- TanStack

Router: A type-safe routing manager for React that handles nested layouts, route loaders, and query parameters dynamically.

- Tailwind CSS (v4): A utility-first style sheet language utilizing CSS variables for styling. It delivers a fast, dark-mode compatible responsive interface across varying phone and tablet screens.

- Radix UI

Primitives: Accessible component primitives (Dialogs, Select fields, Accordions, Dropdown Menus) providing a rich, responsive UX.

- Lucide React: A premium,

vector-based iconography library.

- Recharts: A responsive charting library used to generate run rate charts, wagon wheels, and career progression timelines.

2.2 Native Integration & Storage

- Capacitor JS (v8): The native bridge wrapping the React bundle into an Android package, exposing device file systems and haptic motors.
- Capacitor Communit

y SQLite: Provides native SQLite bindings for Android, allowing the client to execute relational queries directly against the local device storage.

- Capacitor Filesyst

em & Preferences: Manages local settings, authentication tokens, and backup directories for raw database exports.

2.3 Backend API Layer

- Laravel 11: The backend server framework servi ng REST endpoints for roster synchronization, account authentication, and remote backups.

- Laravel Sanctum: Provides lightweight API token authentication for se cure offline-to-online data uploads.

- Laravel Echo & Pusher: Websocket server broadcast framework providing real-time live score updates for public web views

.

- MySQL / MariaDB: The production cloud database structure representing aggregated tenant, club, and match profiles.

3. CricEngine V2 Architecture

3.1 Event-Driven Calculations

CricEngine V2 manages match calculations by maintaining an event log table (`v2_ball_events`). Instead of incrementing total scores incrementally in a single row, every ball is processed as a transaction. This design ensures that errors can be undone, corrected, or recalculated dynamically.

3.2 Sub-Engine Components

3.2.1 InningsEngine

Controls the status of active innings, calculates team runs/wickets, handles first-innings closures, resets scoring parameters for the second-innings, and calculates the exact target score dynamically based on first-innings events.

3.2.2 Undo & Correction Engine

Allows scorers to undo delivery events or edit historical balls (e.g. correcting a run, converting a wide to a legal delivery, changing a batsman dismissal). When a correction occurs, the engine recalculates the entire match snapshot by sequentially replaying remaining ball events in a single SQLite transaction.

3.2.3 MatchCompletionEngine

Fired upon match conclusion, this engine computes the final result string (e.g. "Team A won by 10 runs"), identifies the Player of the Match using a weighted performance algorithm (runs + wickets*20 + catches*10 + boundaries*2), and locks the database records from further modifications.

3.2.4 StatsRecoveryService

A diagnostic layer that runs periodically to audit database records. It scans the raw events log to resolve potential drift, repairing discrepancies in player career stats and historical snapshots.

4. Scorer Operations Manual

4.1 Creating and Starting a Match

- 1. Roster Setup: Ensure players are registered in the global directory. Assign players to respective Match Squads (Dynamic Team Architecture allo ws players to represent different teams over time).

- 2. Match Initialization: Select the two teams, match date, ground, total overs, and extras runs penalties. Choose the toss winner and their decision (Batting first / Bowling first).

- 3. Inital Openers Selection: The app will automatically open selection popups. Pick the opening Str iker, Non-striker, and the opening Bowler to start scoring.

4.2 Recording Deliveries

The scoring board contains a responsive pad with buttons representing runs (0-6) and extras:

- Runs: Tap a digit (0 to 6) to credit runs to the striker and the team. The strike automatically rotates on odd run values.

- Wide / No Ball: Tap "WD" or "NB" to log extras. Select any runs scored on the delivery. Wides do not consume legal deliveries in the ov er.

- Byes / Leg Byes: Tap "BY" or "LB" to credit extras. Runs scored are credited to the team score, but do not increment the batter's career runs.
- Wicket: Tap the "OUT

" button. A dialog prompts you to select the dismissal type (Bowled, Caught, Run Out, Stumped, LBW) and the new incoming batsman.

4.3 Innings Transitions

When the first innings reaches the overs limit or 10 wickets fall, CricLab automatically locks the screen and displays the "Innings 1 Complete" panel. Tapping "Start Next Innings" resets the scoring state machine. The app automatically resets all player states, clears past team players, and prompts you to select the second innings batsmen and bowler.

5. Administrator Operations

5.1 Player Directory Moderation

Only administrators have access to create, modify, and delete players. Global players are independent of team registrations, ensuring dynamic mobility between squads.

- Adding a Player: Input player full name, phone, email, date of birth, preferred batting/bowling styles, and primary role.
- Converting Guest Play

ers: Scorers can add temporary "Guest Players" during match squad creation. Admins can view these guest profiles and merge/convert them into official global players wi th preserved statistics.

5.2 Backup and Recovery Center

Administrators can manage database backups to ensure safety against physical device damage:

- Local Database Exports: Export the entire SQLite database as a `.criclab` JSON archive. This can be copied to external st orage or shared via messenger applications.

- Local Backup Restorations: Import any previous `.criclab` backup file to restore complete player profiles, matches, an d ball events.

- Online Sync: Sync the local database with the server API. The app runs a schema-safe parser to upload and synchronize changes without disru

pti
ng
exi
sti
ng
we
b s
tat
s.